

BREACHLAB

Kendi Kodunu Yaz, Kır, Onar

"İnşa Et → Saldır → Savun" Metodolojisiyle Uygulamalı Bir Proje

Yapı: 6 Aşamalı, Sıralı İlerleyen Proje

Yaklaşım: Yaparak öğrenme — kendi ürününü yaz, test et, güçlendir

Doküman Tipi: Staj Programı ve Proje Planı

İçindekiler

[1. Proje Özeti](#)

[2. Öğrenme Hedefleri](#)

[3. Genel Akış \(Özet\)](#)

[4. Aşamaların Detayı](#)

[Aşama 1 — Temeller \(Yazılım + Sistem\)](#)

[Aşama 2 — Web ve Uygulama Güvenliği Temelleri](#)

[Aşama 3 — Kendi Kırılgan Uygulamasını İnşa Etme](#)

[Aşama 4 — Kendi Uygulamasına Sızma Testi](#)

[Aşama 5 — Güvenli Kodlama ve Savunma Katmanları](#)

[Aşama 6 — Capstone: Bütünleştirme ve Sunum](#)

[5. Kullanılacak Araçlar ve Ortamlar](#)

[6. Değerlendirme ve Mentorluk Süreci](#)

[7. Program Sonrası Öneriler](#)

1. Proje Özeti

"BreachLab", altı aşamadan oluşan tekil ve bütünsel bir proje planıdır. Dağınık konu başlıkları yerine tek bir gerçek dünya senaryosu etrafında kurgulanmıştır: kendi web uygulamanı yazarsın, ona saldırırsın, bulduğun açıkları düzeltirsin ve tüm süreci raporlarsın. Konular ayrı ayrı anlatılmaz; her biri bu tek akışın içinde, sırası geldiğinde ihtiyaç duyduğun için karşına çıkar. Aşamalar sıralıdır — her biri bir öncekinin üzerine inşa edilir — ancak her aşamaya ne kadar zaman ayıracağın, kendi hızına ve derinleşmek istediğin konulara göre esnek tutulur.

Bu yaklaşımın temel gerekçesi şudur: siber güvenlikte gerçekten değer üreten kişiler, sadece saldırı araçlarını kullanabilen değil, kodun neden kırılabilir olduğunu anlayan kişilerdir. Bu nedenle program, yazılım geliştirme ve güvenlik testini birbirinden ayrı iki konu olarak değil, aynı ürünün iki yüzü olarak ele alır.

1.1 Proje Metodolojisi: İnşa Et → Saldır → Savun

- İNŞA ET: Temel bir web uygulaması geliştirirsin (kayıt, giriş, veri paylaşımı içeren basit bir CRUD uygulaması).
- SALDIR: Kendi yazdığın uygulamaya, öğrendiğin güvenlik test yöntemleriyle sızma testi uygularsın.
- SAVUN: Bulduğun açıkları güvenli kodlama teknikleriyle kapatırsın ve süreci raporlarsın.

Bu döngü, "saldırgan" ve "savunmacı" bakış açılarını aynı ürün üzerinde deneyimlemeni sağlar — sektörde bu beceriye genel olarak "security mindset" (güvenlik bakış açısı) denir.

2. Öğrenme Hedefleri

Bu sürecin sonunda aşağıdaki yetkinlikleri kazanmış olursun:

2.1 Teknik Hedefler

- Linux sistem yönetimi (izinler, kullanıcı/servis yönetimi, log inceleme, temel sertleştirme) ve ağ temelleriyle rahat çalışabilme
- Python (veya Flask/Node.js) ile basit fakat işlevsel bir web uygulaması yazabilme
- OWASP Top 10 zafiyet sınıflarını teorik ve pratik düzeyde açıklayabilme
- Burp Suite / ZAP (Zed Attack Proxy) gibi araçlarla manuel test yapabilme
- Güvenli kodlama prensiplerini (input validation, parametrelili sorgular, hashing, secret yönetimi) uygulayabilme
- Temel düzeyde log analizi ve anomali tespiti yapabilme

2.2 Yetkinlik Hedefleri (Soft Skills)

- Bir bulguyu; risk seviyesi, kanıt ve öneri içerecek şekilde profesyonelce raporlayabilme
- Git/GitHub üzerinden düzenli, izlenebilir bir çalışma disiplini geliştirme
- Teknik bir konuyu ekip önünde sunabilme

3. Genel Akış (Özet)

Program altı aşamadan oluşur. Her aşama bir öncekinin üzerine inşa edilir; hiçbir aşama bağımsız bir 'konu' değil, tek projenin bir parçasıdır. Aşamalar arası geçiş, takvime değil, o aşamanın teslimatının tamamlanmış olmasına bağlıdır.

Konu Başlıkları	Uygulama / Görev	Teslimat
Aşama 1 — Temeller (Sanal Makine, Linux, Sistem)	VM kurulumu, Linux'ta derinleşme, mini araç geliştirme	Sertleştirilmiş VM + script + Git geçmişi
Aşama 2 — Web Güvenliği Temelleri, OWASP Top 10	DVWA / Juice Shop üzerinde pratik	Zafiyet bulgu notları
Aşama 3 — Uygulama Geliştirme (kasıtlı kırılğan)	Flask/Node.js ile CRUD uygulaması yazımı	Çalışan mini uygulama + repo
Aşama 4 — Sızma Testi	Kendi uygulamasına Burp Suite/ZAP ile test	Pentest raporu (taslak)
Aşama 5 — Güvenli Kodlama & Savunma	Açıkların kapatılması, loglama eklenmesi	Düzeltilmiş uygulama + önce/sonra dokümanı
Aşama 6 — Capstone: Bütünleştirme ve Sunum	Olay müdahalesi senaryosu + final rapor	Final raporu + sunum

4. Aşamaların Detayı

AŞAMA 1 — Temeller (Yazılım + Sistem)

Amaç: Güvenlik konularına girmeden önce sağlam bir teknik zemin oluşturmak — özellikle Linux tarafında yüzeysel değil, gerçekten elleşerek.

1.1 Sanal Makine Ortamı

Bu aşamaya, kendi izole Linux ortamını kurarak başlarsın. VirtualBox veya VMware üzerine bir Ubuntu Server (veya tercih edersen Kali Linux) sanal makinesi kurarsın. Bu makine, programın geri kalanında hem geliştirme hem de güvenlik testi ortamın olarak kullanılır. Değişiklik yapmadan önce snapshot almayı alışkanlık haline getirirsin — bir şeyi bozarsan geri dönebilmen için.

1.2 Linux'ta Derinleşme

Burada amaç sadece komutları ezberlemek değil; bir sistemin içinde neyin nerede olduğunu, neden öyle çalıştığını ve bunun güvenlik açısından ne anlama geldiğini kavramaktır. Aşağıdaki konuların her biri, ileride sızma testi ve savunma aşamalarında doğrudan işine yarayacak.

Konu Başlıkları	Uygulama / Görev	Teslimat
Dosya sistemi hiyerarşisi, izinler (chmod/chown), SUID/SGID biti	Farklı izin senaryoları kurup dosya erişimini test etme	İzin haritası + SUID'li dosya taraması notu
Kullanıcı/grup yönetimi, sudo yetkilendirme, /etc/passwd ve /etc/shadow	Yeni kullanıcı/grup oluşturma, sudoers dosyasını düzenleme	Kullanıcı yönetimi notu
Süreç ve servis yönetimi: ps, top, systemctl, journalctl	Servisleri başlatma/durdurma/inceleme, açık süreçleri listeleme	Servis ve süreç inceleme notu
Paket yönetimi (apt/dpkg) ve sistem güncelleme mantığı	Paket kurma/kaldırma, kurulu paketleri denetleme	Kurulu paket envanteri
Log dosyaları ve güvenlik izleri: /var/log, auth.log, cron logları	Log dosyalarında başarısız giriş denemesi / şüpheli aktivite arama	Log inceleme notu
Temel sertleştirme (hardening): SSH ayarları, ufw ile port kısıtlama, gereksiz servisleri kapatma	SSH'ı güvenli hale getirme, ufw ile basit bir güvenlik duvarı kurma	Sertleştirilmiş VM + yapılan değişikliklerin notu

Bu bölümdeki her konu bilinçli seçildi: dosya izinleri ve SUID biti, ileride kendi uygulamanda ve sızma testinde karşına çıkacak yetki yükseltme (privilege escalation) mantığının temelidir; log inceleme becerisi, Aşama 5'teki anomali tespiti çalışmasının doğrudan zemini; SSH sertleştirme ise "güvenli sistem kurmak" ile "güvenlik açığı bırakmak" arasındaki farkı ilk elden gösterir.

1.3 Git, Python ve Ağ Temelleri

Konu Başlıkları	Uygulama / Görev	Teslimat
Git & GitHub: branch, commit, PR disiplini	Kişisel repo açma, düzenli commit alışkanlığı	İlk GitHub reposu

Konu Başlıkları	Uygulama / Görev	Teslimat
Python temelleri: script yazma, dosya I/O, requests	Basit otomasyon scriptleri yazımı	3-4 küçük Python scripti
Ağ temelleri: TCP/IP, DNS, HTTP/HTTPS, portlar, OSI	Wireshark ile trafik gözlemleme	Port tarayan mini Python aracı

Bu aşamanın sonunda, kendi kurduğun sertleştirilmiş VM'i, yazdığın basit bir port tarama scriptini ve düzenli commit geçmişi olan bir Git reposunu mentörüne sunarsın.

AŞAMA 2 — Web ve Uygulama Güvenliği Temelleri

Amaç: OWASP Top 10'u teoride öğrenip kırılğan bir ortamda pratiğe dökmek.

Konu Başlıkları	Uygulama / Görev	Teslimat
HTTP metodları, cookie/session, auth mekanizmaları, JWT	Tarayıcı geliştirici araçlarıyla istek/yanıt inceleme	Auth akışı diyagramı
OWASP Top 10: SQLi, XSS, CSRF, SSRF, broken auth	DVWA / OWASP Juice Shop üzerinde egzersizler	Zafiyet başına kısa bulgu notu
Burp Suite / ZAP tanıtımı ve temel kullanım	Proxy ayarları, request interception, repeater kullanımı	Yakalanmış örnek istek/yanıt kayıtları
Zafiyetlerin kök neden analizi (neden oluşur?)	Her zafiyet için 'bu neden çalıştı' açıklaması yazımı	Konsolide OWASP Top 10 bulgu raporu

Bu aşamanın çıktısı olan bulgu raporu, Aşama 3'te yazılacak uygulamanın hangi açıkları 'kasıtlı olarak' içereceğine temel oluşturur.

AŞAMA 3 — Kendi Kırılğan Uygulamasını İnşa Etme

Amaç: Zafiyetlerin kod seviyesinde nasıl ve neden oluştuğunu, onları bizzat yazarak kavramak.

Konu Başlıkları	Uygulama / Görev	Teslimat
Proje mimarisi planlama (Flask veya Node.js seçimi)	Kullanıcı kayıt/giriş modülü geliştirme	Çalışan auth modülü
CRUD işlevleri: not/mesaj paylaşma özelliği	Veritabanı bağlantısı, temel CRUD endpoint'leri	Çalışan CRUD uygulaması
Kasıtlı zafiyet ekleme (mentor onayıyla)	SQL injection, zayıf hash, hardcoded secret ekleme	Zafiyetli kod + gerekçe notu
Kod review süreci	Git PR açma, mentor geri bildirimi alma	README + PR geçmişi

Bu aşamada amaç 'güvenli kod yazmak' değil, tam tersine; bir geliştiricinin hangi alışkanlıkları nedeniyle açık bıraktığını içeriden görmektir. Bu, ileride kod incelemesi yaparken sana gerçekçi bir sezgi kazandırır.

AŞAMA 4 — Kendi Uygulamasına Sızma Testi

Amaç: Saldırgan bakış açısını, üzerinde tam hakimiyet olan bir hedefte uygulamak.

Konu Başlıkları	Uygulama / Görev	Teslimat
Test kapsamının belirlenmesi (scope), keşif (recon)	Nmap ile kendi lab ortamında port/servis taraması	Recon notları
Manuel test: auth bypass, SQLi, XSS denemeleri	Burp Suite ile kendi uygulamasına saldırı	Bulgu listesi (kanıt/ekran görüntüsü ile)
Bulguların risk seviyelendirmesi (CVSS mantığı)	Her bulgu için etki/olasılık değerlendirmesi	Risk skorlu bulgu tablosu
Pentest raporu yazımı	Bulgu, kanıt, risk, öneri formatında rapor hazırlama	Taslak pentest raporu

Bu aşamanın en kritik kazanımı, bir güvenlik açığını bulmakla onu 'iş değeri olan bir bulgu' haline getirmek arasındaki farkı öğrenmektir.

AŞAMA 5 — Güvenli Kodlama ve Savunma Katmanları

Amaç: Bulunan açıkları düzelterip güvenlik döngüsünü tamamlamak; savunma tarafını da deneyimlemek.

Konu Başlıkları	Uygulama / Görev	Teslimat
Parametrelili sorgular, input validation, output encoding	SQLi ve XSS açıklarının kod seviyesinde kapatılması	Güncellenmiş, düzeltilmiş kod
Güvenli hash (bcrypt/argon2), secret yönetimi (.env)	Şifreleme mekanizmasının yeniden yazılması	Güncellenmiş auth modülü
Rate limiting, temel loglama ekleme	Basit bir log mekanizması kurulması	Log üreten uygulama
Log analizi ve anomali tespiti egzersizi	Verilen örnek log setinde şüpheli aktivite arama	Anomali tespit notu

Bu aşamanın sonunda, Aşama 4'te bulduğun her açığın karşısına 'nasıl kapattım' notunu yazarak önce/sonra karşılaştırmalı bir doküman oluşturursun.

AŞAMA 6 — Capstone: Bütünleştirme ve Sunum

Amaç: Tüm süreci tek bir anlatıda birleştirip aktarabilmek — raporlama ve iletişim de bu işin parçasıdır.

Konu Başlıkları	Uygulama / Görev	Teslimat
Tüm sürecin (inşa-saldırı-savunma) tek dokümanda toplanması	Final raporu taslağının hazırlanması	Taslak final raporu
Olay müdahalesi (Incident Response) mini senaryosu	Kurgusal log seti üzerinden 'ne oldu' analizi	Olay analizi notu
Sunum hazırlığı	Süreç, bulgular ve öğrenilenlerin sunum formatına	Sunum dosyası

Konu Başlıkları	Uygulama / Görev	Teslimat
	dökülmesi	(taslak)
Final sunumu ve değerlendirme	Ekip önünde demo ve sunum	Final raporu + sunum + genel değerlendirme

5. Kullanılacak Araçlar ve Ortamlar

5.1 Sanal Ortam ve Sistem

- VirtualBox veya VMware Workstation Pro (kişisel kullanım için ücretsiz) — sanal makine ortamı
- Ubuntu Server veya Kali Linux (VM işletim sistemi)

5.2 Geliştirme

- Python 3 / Flask veya Node.js / Express
- Git & GitHub
- VS Code
- Docker (opsiyonel — ortam izolasyonu için)

5.3 Güvenlik Testi

- OWASP Juice Shop / DVWA (kasıtlı kırılabilir lab ortamları)
- Burp Suite Community Edition
- ZAP (Zed Attack Proxy — eski adıyla OWASP ZAP, artık bağımsız bir proje olarak sürdürülüyor)
- Nmap
- Wireshark (opsiyonel, ağ trafiği analizi için)

Tüm araçlar ücretsizdir ve kurulumu kolaydır; bu nedenle programın önünde bir lisans/maliyet engeli yoktur.

6. Değerlendirme ve Mentorluk Süreci

- Düzenli mentor check-in'leri: her aşamanın sonunda, o aşamada anlaşılan/anlaşılmayan konuların gözden geçirilmesi.
- Her aşama sonunda somut bir teslimat (kod, rapor veya doküman) beklenir — bu, ilerlemenin ölçülebilir olmasını sağlar.
- Kod review'lar Git PR üzerinden yapılır; geri bildirim yazılı olarak kalıcı hale getirilir.
- Final değerlendirmesi; teknik derinlik, rapor kalitesi ve sunum becerisi olmak üzere üç boyutta yapılır.

7. Program Sonrası Öneriler

Programı tamamladığında, ilgi duyduğun yöne göre aşağıdaki yönlerden biriyle devam etmen önerilir:

- CTF (Capture the Flag) tarzı platformlar: TryHackMe veya HackTheBox'ın 'beginner' path'leri.
- Daha derin bir uygulama güvenliği (AppSec) uzmanlaşması: kod tabanlı statik analiz araçları (SAST) ile tanışma.
- Ağ güvenliği / SOC analistliği yönünde ilerlemek isterse: temel SIEM araçları ve log yönetimi konularına geçiş.

Bu doküman, ilerleyen dönemlerde gösterdiği ilgiye göre güncelleyebileceğin bir yol haritası olarak kullanılabilir.